Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по индивидуальной практической работе № 2

по дисциплине
«ИТиВП»

на тему «Деплой и мониторинг приложения»

Выполнил: студент группы 320601
А. С. Образцов

Проверила: Н. В. Хаджинова

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью работы является настройка автоматической сборки и развёртывания серверной части сервиса планирования задач по методу *GTD*, её запуск на бесплатной облачной платформе и подключение мониторинга: проверки доступности, наблюдения за ресурсами, централизованных журналов и сбора ошибок с уведомлениями. Основой служит *REST API* на *Node.js* и *Express*, разработанный в предыдущих работах.

Приложение доступно по адресу <TODO: продакшен-URL>, исходный код — в репозитории *https://github.com/Alexandr-Obraztsov/gtd-task-service*, ветка *ipr22*. Ссылки на дашборды мониторинга и ошибок приведены в разделе «Мониторинг и деплой» файла *README.md*; в черновике отчёта они остаются заглушками, поскольку аккаунты внешних сервисов ещё не созданы.

2 ХОД РАБОТЫ

2.1 Выбор инструментов

Инструменты выбирались по трём признакам: бесплатный тариф, доступность из Беларуси и возможность настроить всё из репозитория, не прибегая к ручным действиям при каждом выпуске. Выбранные средства сведены в таблице 2.1.

Таблица 2.1 – Выбранные инструменты

Задача	Инструмент	Обоснование
CI/CD	*GitHub Actions*	репозиторий уже на *GitHub*, секреты и окружения встроены, достаточно бесплатных минут
Бэкенд	*Render* (*Docker*)	принимает *Dockerfile*, запускает деплой по *Deploy Hook*, конфигурация хранится в *render.yaml*
Фронтенд	*Vercel*	деплой собранной статики одной командой *CLI*
База данных	*Neon*	управляемый *PostgreSQL* с *TLS* на бесплатном тарифе
Доступность	*UptimeRobot*	проверка раз в пять минут, публичная страница статуса, оповещения в *Telegram*
Журналы	*Better Stack* (*Logtail*)	подключается как ещё один транспорт *winston*, поиск по полям *JSON*
Ошибки	*Sentry*	стек вызова, окружение, релиз, группировка повторяющихся ошибок
Уведомления	*Telegram Bot API*	один запрос *curl* из *workflow*, без сторонних расширений

2.2 Проверка качества кода: *ESLint* и тесты

Для статического анализа добавлен *ESLint* с конфигурацией *eslint.config.js* (рекомендованные правила, запрет *console*, строгое сравнение, *prefer-const*) и скрипт *npm run lint*. Прогон линтера по всему проекту завершился без замечаний, результат представлен на рисунке 2.1.

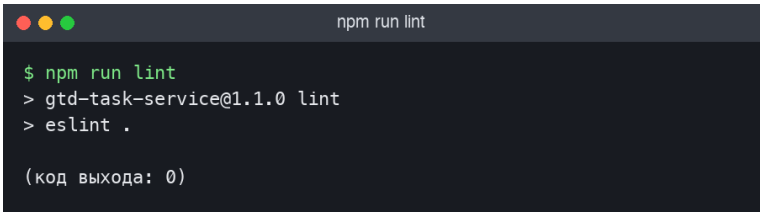


Рисунок 2.1 – Результат *npm run lint*

Автоматические тесты написаны на *Jest* и *supertest*. Они запускают настоящее приложение (*createApp()*) с реальной базой данных, а не заглушки. Проверяются: */health* и */health/ready*, ответ 401 без токена и с поддельным *JWT*, валидация (слабый пароль, лишнее поле, битый *JSON*, неверный *Content-Type*), доступ к */metrics* для разных ролей, счётчик запросов, политика *CSP* страницы */dashboard* и маршрут тестовой ошибки. Всего 40 тестов в шести файлах. Список тестов представлен на рисунках 2.2 и 2.3.

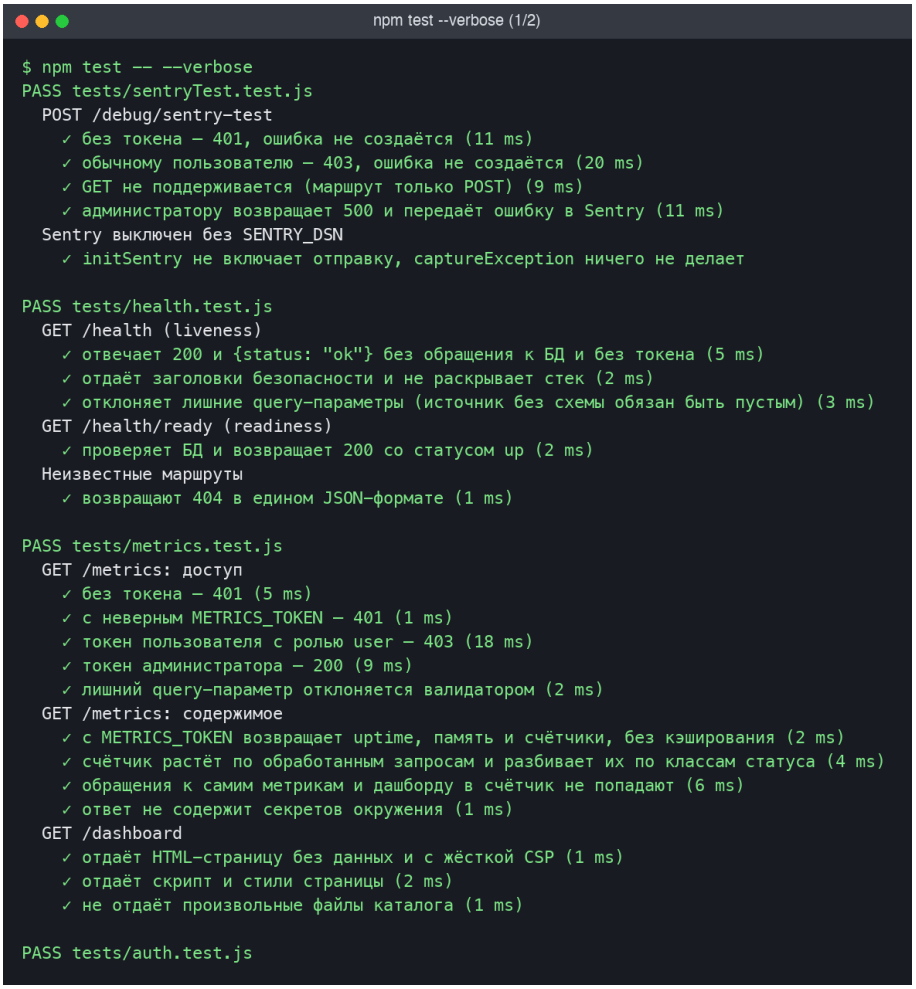


Рисунок 2.2 – Выполнение тестов (начало)

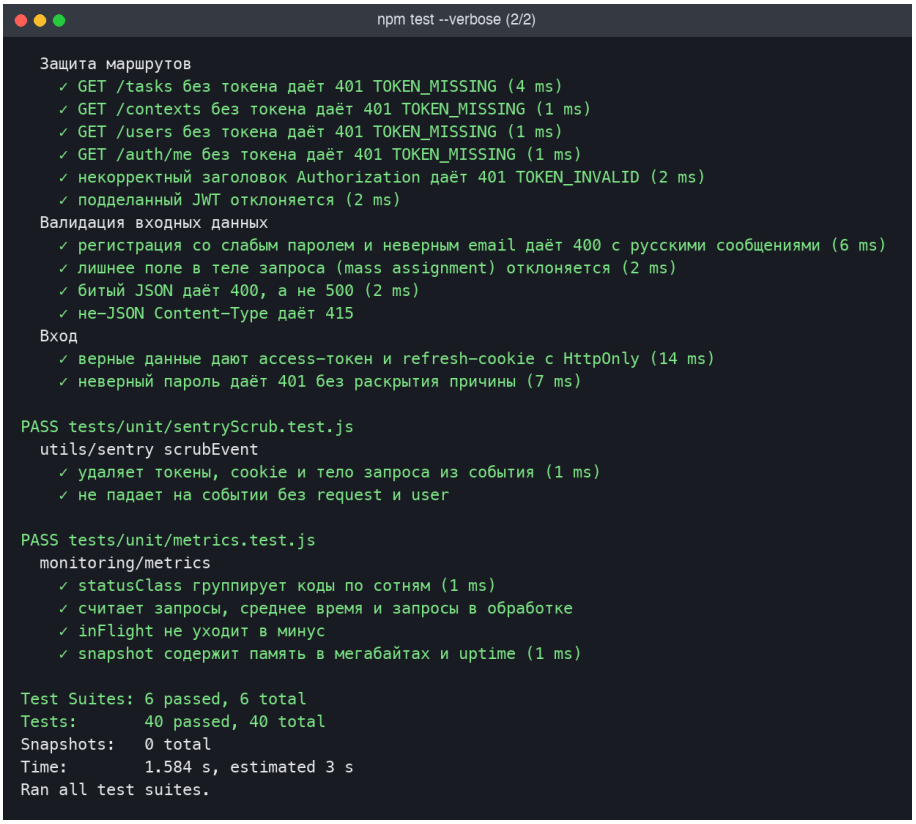


Рисунок 2.3 – Выполнение тестов (окончание)

2.3 Контейнеризация

Образ описан в *Dockerfile* из двух стадий: в первой ставятся только production”=зависимости (*npm ci –omit=dev* с кэшем *npm*), во второй копируются исходники, и процесс запускается от пользователя *node*, а не *root*. Файл *.dockerignore* исключает из контекста *.env*, *node_modules*, журналы, тесты и отчёты, поэтому секреты не могут попасть в образ. Проверка живости встроена в образ инструкцией *HEALTHCHECK*:

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s \
  --retries=3 \
  CMD node -e ”fetch(’http://127.0.0.1:’+(process.env.PORT\
||10000)+’/health’).then(r=>process.exit(r.ok?0:1))\
.catch(()=>process.exit(1))”
```

Образ собран локально и запущен против контейнера с базой данных: состояние контейнера стало *healthy*, а */health/ready* вернул *200*. Размер образа составил 417 МБ. Вывод команд представлен на рисунке 2.4.

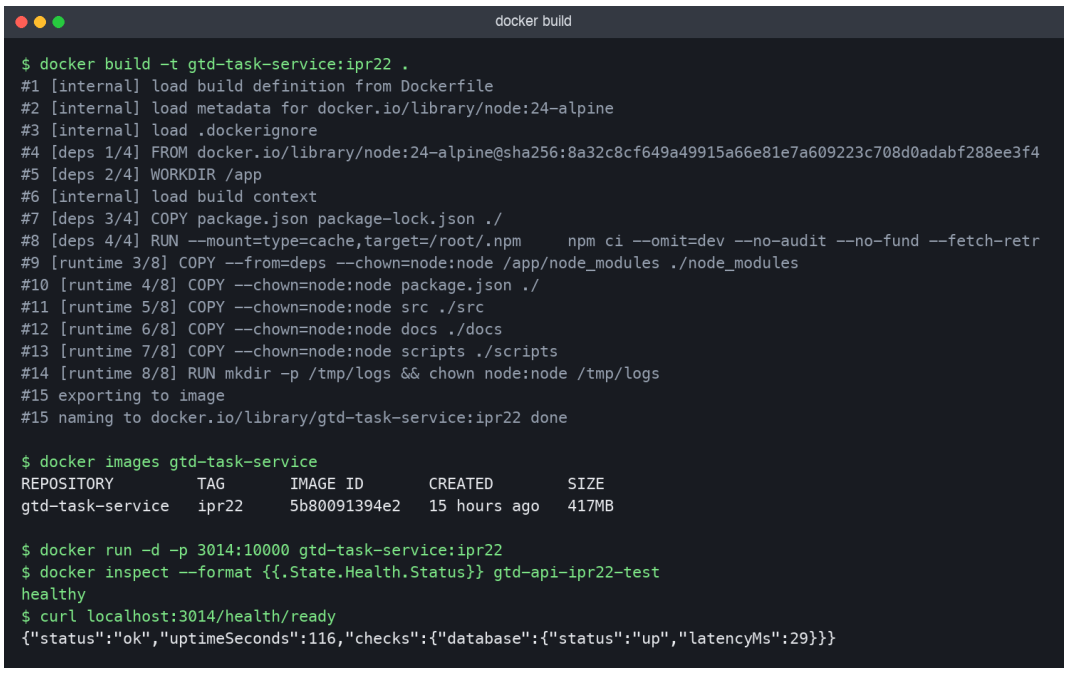


Рисунок 2.4 – Сборка и запуск образа *Docker*

2.4 *Workflow GitHub Actions*

Файл *.github/workflows/deploy.yml* запускается при *push* в ветку *main*, при *pull request* и вручную. Для *pull request* выполняются только проверки. Начало файла с заданиями *test* и *docker* представлено на рисунке 2.5. Задание *test* поднимает *PostgreSQL 16* как *service container* с проверкой готовности, ставит зависимости, запускает линтер и тесты. Задание *docker* собирает образ через *buildx* с кэшем слоёв *type=gha*, не публикуя его.

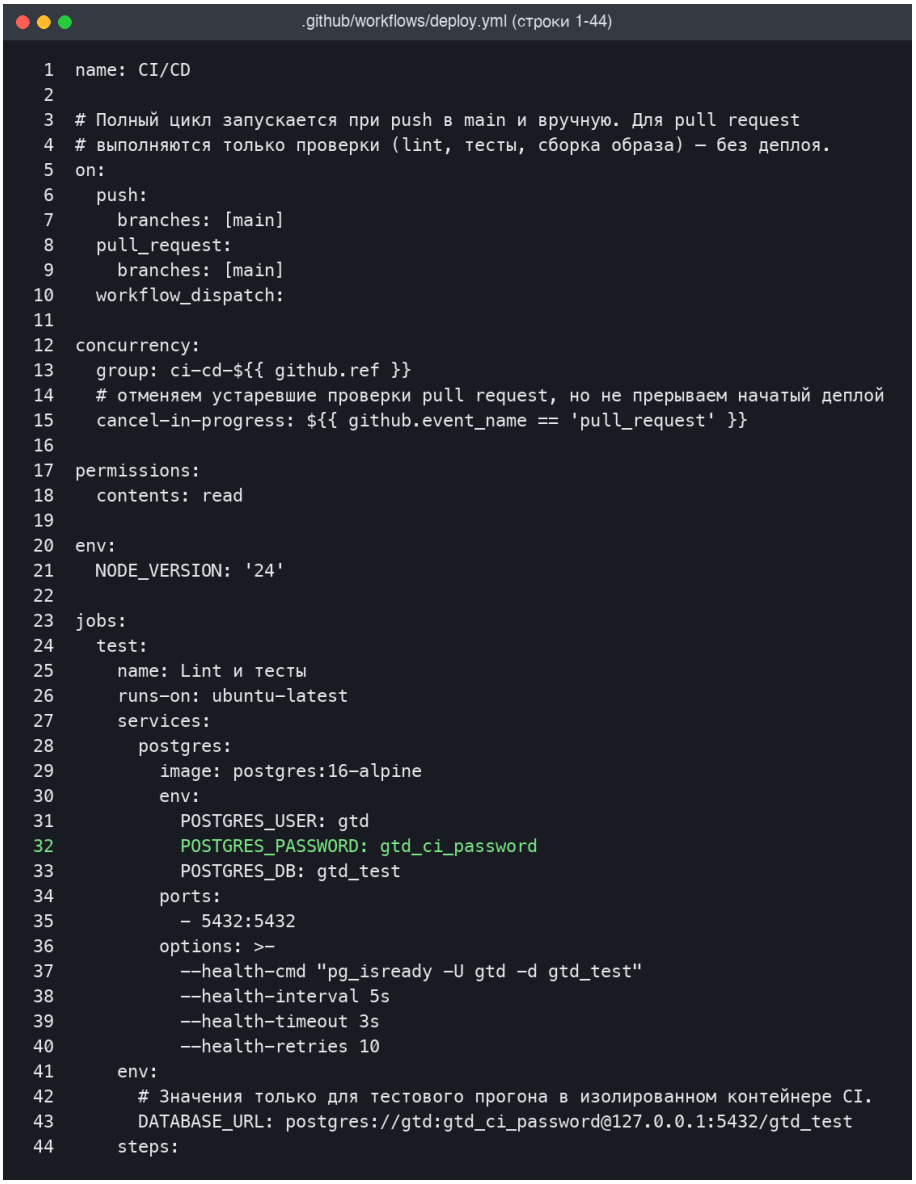


Рисунок 2.5 – Файл *workflow*: триггеры и задания *test*, *docker*

Задания деплоя и уведомления представлены на рисунке 2.6. Деплой бэкенда выполняется только после успеха проверок: запрос *POST* на адрес *Deploy Hook* из секрета, затем опрос */health* до десяти минут. Деплой фронтенда на *Vercel* условный: он идёт, лишь если в репозитории есть *client/package.json* и заданы секреты *VERCEL_**. Оба деплоя и уведомление сами пропускаются, если нужных секретов нет, поэтому *workflow* не падает в форке без настроенных сервисов.



Рисунок 2.6 – Файл *workflow*: задания деплоя

Страница запуска *workflow* в разделе *Actions* репозитория: [скриншот будет добавлен после настройки сервиса].

2.5 Секреты и переменные

Значения секретов в репозитории не хранятся. Для *workflow* требуются секреты *RENDER_DEPLOY_HOOK_URL*, *VERCEL_TOKEN*, *VERCEL_ORG_ID*, *VERCEL_PROJECT_ID*, *TELEGRAM_BOT_TOKEN*, *TELEGRAM_CHAT_ID* и переменная *PRODUCTION_URL*; их назначение и источник получения описаны в *README.md*. Секреты самого приложения (*DATABASE_URL*, *JWT_SECRET*, *JWT_REFRESH_SECRET*, *METRICS_TOKEN*, *SENTRY_DSN*, *LOGTAIL_SOURCE_TOKEN*) вводятся только в панели *Render*: в *render.yaml* они помечены *sync: false*, чтобы значения не попали в репозиторий.

2.6 Деплой на *Render* и *Vercel*

Конфигурация *Render* хранится в *render.yaml*: сервис типа *web* с рантаймом *docker*, бесплатный план, регион *frankfurt*, *healthCheckPath: /health*. Автодеплой выключен (*autoDeploy: false*), чтобы сборку запускал только *workflow* после успешных проверок. База данных вынесена в *Neon*; приложение подключается к ней по *TLS* (переменная *DATABASE_SSL=true*, проверка сертификата включена). Фрагмент конфигурации:

```
services:
  - type: web
    name: gtd-task-service
    runtime: docker
    plan: free
    region: frankfurt
    autoDeploy: false
    healthCheckPath: /health
    envVars:
      - key: NODE_ENV
        value: production
      - key: DATABASE_URL
        sync: false
```

Для фронтенда добавлен *vercel.json*: команды установки и сборки направлены в каталог *client*, выходной каталог — *client/dist*, все пути, кроме */assets/*, перенаправляются на *index.html*, чтобы работала маршрутизация одностраничного приложения, добавлены заголовки безопасности. Страница деплоя на платформе *Render*: [скриншот будет добавлен после настройки сервиса]. Страница проекта на *Vercel*: [скриншот будет добавлен после настройки сервиса].

2.7 Мониторинг доступности

Маршрут *GET /health* служит проверкой живости: он отвечает *{"status":"ok"}* без обращения к базе данных, поэтому медленная или недоступная база не приводит к перезапуску контейнера. Новый маршрут *GET /health/ready* проверяет готовность: выполняет *SELECT 1* с таймаутом 3 с и возвращает *200* либо *503*. Причина сбоя драйвера наружу не отдаётся, поскольку текст ошибки может содержать адрес базы. Ответы обоих маршрутов, а также отказ */metrics* без токена представлены на рисунке 2.7.

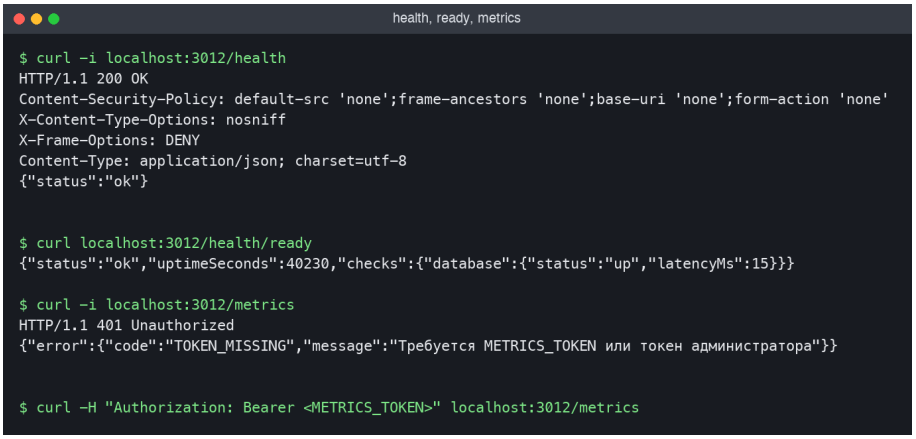


Рисунок 2.7 – Ответы */health*, */health/ready* и */metrics* без токена

Для *UptimeRobot* в *README.md* приведена инструкция: монитор *HTTP(s)* на */health* с интервалом пять минут и ключевым словом *”status”:”ok”*, канал оповещений — *Telegram*, публичная страница статуса. Монитор на */health/ready* предлагается ставить с большим интервалом, потому что он обращается к базе и не даёт ей засыпать. Дашборд *UptimeRobot*: [скриншот будет добавлен после настройки сервиса].

2.8 Метрики и дашборд

Промежуточный обработчик *metricsCollector* считает запросы: общее число, находящиеся в обработке, разбивку по классам статуса и среднее время ответа. Маршрут *GET /metrics* возвращает также время работы процесса, *process.memoryUsage()* в мегабайтах, среднюю нагрузку и свободную память системы. Ответ для запроса с токеном представлен на рисунке 2.8.



Рисунок 2.8 – Ответ *GET /metrics*

Метрики раскрывают версию *Node.js*, объём памяти, нагрузку и профиль трафика, то есть материал для разведки, поэтому доступ ограничен. Он открыт по статическому токену *METRICS_TOKEN* (его выдают сборщику метрик или дашборду) либо по *JWT* пользователя с ролью *admin*. Токен сравнивается через *crypto.timingSafeEqual* по хешам *SHA-256*, чтобы время ответа не выдавало совпавшие символы; неверные попытки попадают в журнал безопасности и ограничены лимитером. Маршрут */health* оставлен открытым намеренно: он нужен внешнему монитору и не содержит данных.

Страница */dashboard* обновляется автоматически (5, 10 или 30 секунд). Приложение отдаёт заголовок *CSP default-src 'none'*, а валидатор требует пустые *params*, *query* и *body* у маршрутов без схемы. Поэтому страница получила собственную политику (*script-src 'self'*, *connect-src 'self'*, без *unsafe-inline*): скрипт и стили вынесены в отдельные файлы, данные выводятся через *textContent*, а не через *innerHTML*. Токен вводится в форму и хранится в *sessionStorage* вкладки, в адрес он не попадает. Обращения к самим метрикам

в счётчик не входят, иначе автообновление искажало бы статистику. Дашборд после ввода токена, на фоне тестовой нагрузки, представлен на рисунке 2.9.

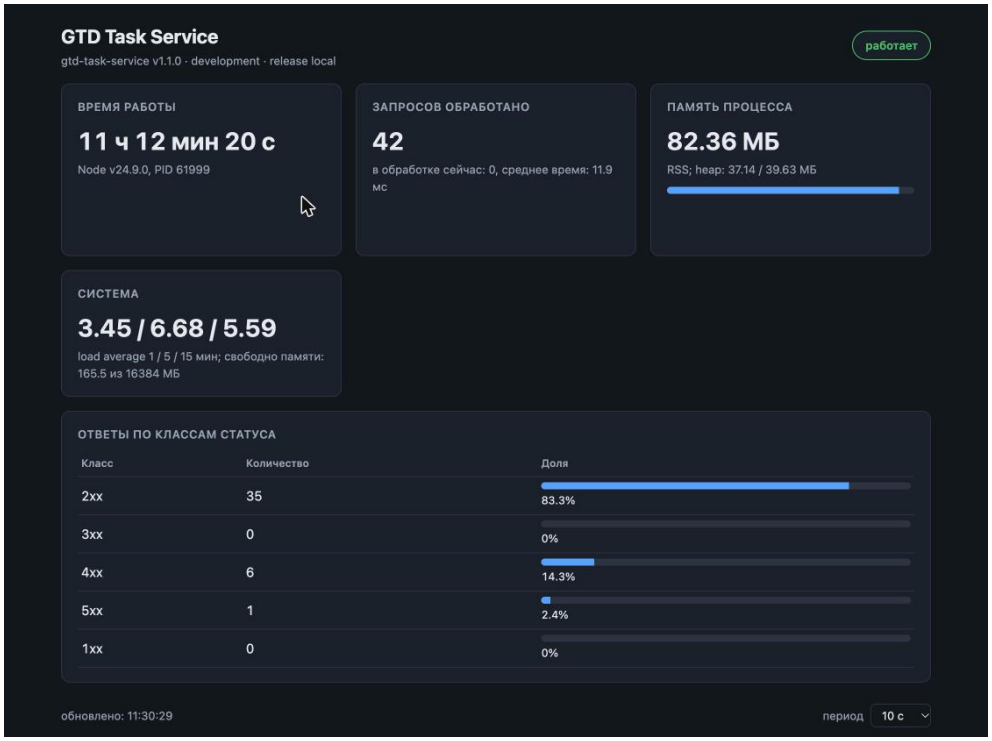


Рисунок 2.9 – Страница */dashboard* в браузере

2.9 Журналирование

Winston уже писал журналы в консоль и в файлы с ротацией. Добавлен транспорт *@logtail/winston*, который включается только при наличии *LOGTAIL_SOURCE_TOKEN*; без токена поведение прежнее:

```
const logtailClient = config.logtail.sourceToken
  ? new Logtail(config.logtail.sourceToken,
      config.logtail.endpoint
        ? { endpoint: config.logtail.endpoint } : {})
  : null;

function remoteTransports() {
  return logtailClient
    ? [new LogtailTransport(logtailClient, { level: 'info' })]
    : [];
}
```

К каждой записи добавлены поля *service* и *release*. Фиксируются запуск сервера (порт, окружение, включённые *Sentry*, удалённые журналы и метрики), каждый запрос (для кодов от 400 уровень *warn*), отклонённые запросы

с перечнем полей, не прошедших валидацию (значения не пишутся), события безопасности и ошибки со стеком. Необработанные *uncaughtException* и *unhandledRejection* записываются, отправляются в *Sentry*, после чего процесс завершается, а платформа перезапускает контейнер. Локальные записи после тестовой нагрузки представлены на рисунке 2.10.

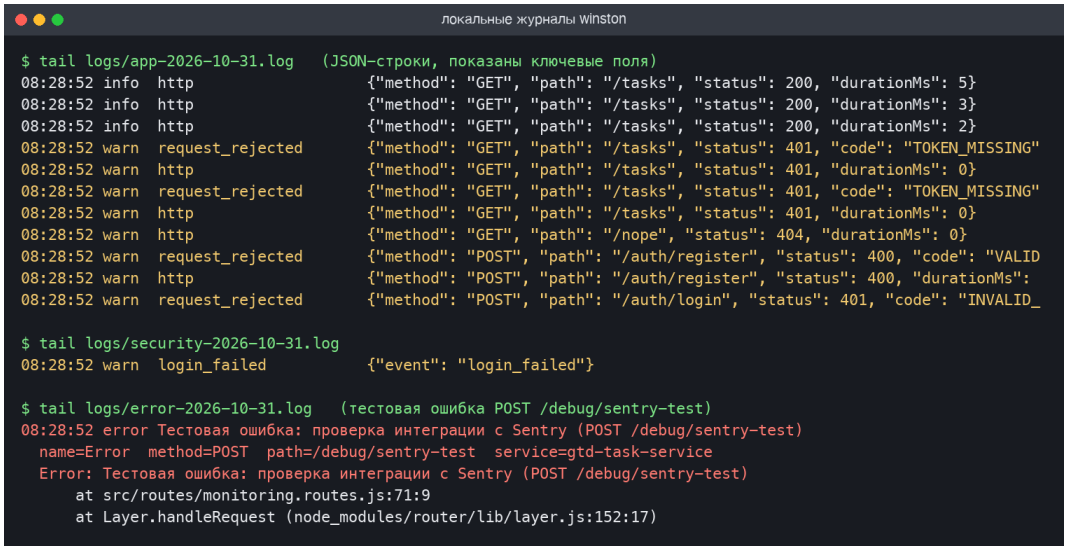


Рисунок 2.10 – Локальные журналы приложения

Интерфейс *Better Stack* с поступившими записями: [скриншот будет добавлен после настройки сервиса].

2.10 Сбор ошибок: *Sentry*

Модуль *src/utils/sentry.js* инициализирует *@sentry/node* только при наличии *SENTRY_DSN*. Перед отправкой событие очищается: удаляются заголовки *Authorization* и *Cookie*, тело запроса и адрес пользователя, остаётся только его *id*:

```
function scrubEvent(event) {
  if (event.request) {
    for (const name of Object.keys(event.request.headers || {})) {
      if (SENSITIVE_HEADERS.includes(name.toLowerCase())) {
        delete event.request.headers[name];
      }
    }
    delete event.request.cookies;
    delete event.request.data;
  }
  if (event.user) event.user = { id: event.user.id };
```

```
  return event;
}
```

Обработчик ошибок приложения передаёт в *Sentry* все ошибки со статусом от 500, а клиентские ошибки 4xx не отправляет, чтобы не засорять список. Для проверки интеграции добавлен маршрут *POST /debug/sentry-test*. Он безопасен: доступен только роли *admin*, только методом *POST*, не принимает данных, а клиенту возвращает обычный ответ 500 без стека. Маршрут оставлен и в *production*, иначе подключение нельзя проверить на живом сервисе.

Внешнего аккаунта *Sentry* нет, поэтому интеграция проверена на локальном приёмнике: значение *SENTRY_DSN* указывало на заглушку *127.0.0.1*, которая принимала запросы. Событие с тестовой ошибкой дошло до приёмника с окружением, релизом и идентификатором пользователя, без заголовка *Authorization*; журналы *@logtail/winston* дошли с заголовком *Bearer*. Вывод проверки представлен на рисунке 2.11. Это проверка кода, а не снимок настоящего сервиса.



Рисунок 2.11 – Проверка отправки событий на локальный приёмник

Дашборд *Sentry* с примером тестовой ошибки: [скриншот будет добавлен после настройки сервиса].

Для *React*”=клиента (ветка с каталогом *client*) в *README.md* описана установка *@sentry/react*: вызов *Sentry.init* до рендера, компонент *Sentry.ErrorBoundary* вокруг корня приложения, отдельный *DSN* в переменной *VITE_SENTRY_DSN* и загрузка карт исходного кода плагином *@sentry/vite-plugin*. В этой работе клиентский код не менялся.

2.11 Уведомления в *Telegram*

Последнее задание *workflow* (*notify*) выполняется всегда, даже если предыдущие задания упали, определяет итог по результатам всех заданий и отправляет сообщение запросом *curl* к *Bot API*. Данные, которые могут содержать произвольные символы (имя ветки, автор), передаются через переменные окружения, а не подставляются в текст команды, что исключает внедрение команд:

```
API=https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}
curl --fail --silent --show-error --max-time 30 \
  "${API}/sendMessage" \
  --data-urlencode "chat_id=${TELEGRAM_CHAT_ID}" \
  --data-urlencode "text=${TEXT}" > /dev/null
```

Итог (успех, провал или отмена) определяется оператором *case* по строке результатов всех заданий (*needs.*.result*).

Шаг пропускается, если секреты бота не заданы. Пример полученного уведомления: [скриншот будет добавлен после настройки сервиса].

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Список контрольных вопросов составлен автором работы самостоятельно по теме задания.

1 *Что такое CI/CD? CI* (непрерывная интеграция) — автоматическая сборка и проверка каждого изменения: линтер, тесты, сборка образа. *CD* (непрерывная доставка или развёртывание) — автоматическая выкладка проверенной сборки в окружение. Вместе они сокращают время от коммита до работающей версии и убирают ручные, ошибкоопасные шаги.

2 *Как устроен workflow GitHub Actions?* Файл *YAML* в *.github/workflows* описывает события запуска (*push*, *pull_request*, *workflow_dispatch*) и задания (*jobs*). Каждое задание выполняется на отдельном исполнителе (*runner*) и состоит из шагов: готовых действий (*uses*) или команд (*run*). Порядок задаёт ключ *needs*, условия — *if*.

3 *Как хранить секреты и почему не в репозитории?* В *GitHub Secrets* (шифруются, маскируются в журнале) и в панели платформы. Попавший в историю *git* секрет нельзя считать секретом: репозиторий публичный, история хранится вечно. Поэтому в репозитории только *.env.example*, а в *render.yaml* значения заменены на *sync: false*.

4 *Зачем в тестах service container?* Он поднимает настоящий *PostgreSQL* рядом с заданием, поэтому тесты проверяют реальные запросы и ограничения, а не заглушку. Контейнер создаётся чистым в каждом запуске, что исключает влияние данных прошлых прогонов.

5 *Зачем нужны Docker, многостадийная сборка и .dockerignore?* Образ даёт одинаковую среду на машине разработчика, в *CI* и на сервере. Многостадийная сборка оставляет в итоговом образе только нужное, запуск не от *root* снижает ущерб при взломе, а *.dockerignore* не пускает в образ секреты и мусор.

6 *Как кэш ускоряет сборку? actions/setup-node* с *cache: npm* сохраняет загруженные пакеты между запусками, а *buildx* с *type=gha* — слои образа. Неизменившиеся слои (зависимости) берутся из кэша, пересобирается только код. Ключом кэша служат хеши *package-lock.json* и инструкций *Dockerfile*.

7 *Как работает деплой на Render и Vercel? Render* собирает образ из репозитория по *Dockerfile* и заменяет работающий экземпляр новым после успешной проверки */health*; сборку запускает *Deploy Hook*, то есть секретный *URL*, на который *workflow* отправляет *POST*. *Vercel* принимает готовую сборку статики командой *vercel deploy –prebuilt* и раздаёт её через *CDN*.

8 *Чем проверка живости отличается от проверки готовности?* Живость (*/health*) отвечает, работает ли процесс; при провале платформа перезапускает контейнер. Готовность (*/health/ready*) отвечает, может ли сервис обслуживать запросы, то есть доступна ли база; при провале трафик временно

не направляют. Если включить в проверку живости базу, её сбой вызовет бессмысленные перезапуски приложения.

9 *Как работает внешний монитор UptimeRobot?* Раз в пять минут он запрашивает адрес из внешней сети, проверяет код ответа и ключевое слово, при сбое шлёт оповещение (в *Telegram*) и копит статистику доступности для публичной страницы. Ложные срабатывания уменьшает повторная проверка. Внешний монитор видит то, что видит пользователь, в отличие от проверки изнутри сети.

10 *Зачем нужны централизованные журналы?* Файловая система контейнера на *Render* временная, и локальные файлы пропадают при перезапуске. Внешний сервис хранит записи, позволяет искать по полям, фильтровать по уровню и строить оповещения. В журналы нельзя писать пароли, токены и значения полей запроса; поэтому при ошибках валидации пишутся только имена полей.

11 *Чем Sentry отличается от журналов?* Журнал — поток всех событий, *Sentry* — каталог проблем: он группирует одинаковые ошибки, показывает стек, частоту, первую и последнюю дату, окружение и релиз, позволяет назначить ответственного и пометить исправленной. Данные перед отправкой очищаются (*beforeSend*).

12 *Как безопасно сделать эндпоинт для тестовой ошибки?* Ограничить доступ ролью *admin*, разрешить только *POST*, не принимать входных данных, не раскрывать причину клиенту и записывать обращение в журнал. Альтернативы — включать маршрут только вне *production* или по флагу окружения, но тогда проверить интеграцию на живом сервисе нельзя.

13 *Какие метрики собираются и как они защищены?* Время работы, память процесса, нагрузка и память системы, число запросов по классам статуса и среднее время ответа. Доступ — по *METRICS_TOKEN* или роли *admin*, токен сравнивается за постоянное время, неверные попытки журналируются и ограничиваются. Для нескольких экземпляров счётчики в памяти потребуют внешнего хранилища, например *Prometheus*.

14 *Какие существуют стратегии отката?* Откат к предыдущему образу (на *Render — Rollback* в истории деплоев, на *Vercel — Promote* прошлого деплоя), *blue-green* (два окружения и переключение трафика) и *canary* (новая версия получает долю трафика). Откат кода безопасен, пока изменения схемы базы только добавляют столбцы; в проекте миграций нет, схема создаётся *sync()*.

15 *Какие ограничения у бесплатных тарифов? Render* усыпляет сервис через 15 минут без трафика, холодный старт занимает до минуты, часы ограничены; *Neon* отключает вычисления при простое и ограничивает объём; *Sentry*, *Better Stack* и *UptimeRobot* ограничивают события, объём журналов и число

мониторов. Смягчение: пинг монитором, допустимая задержка первого ответа, отбрасывание излишних данных.

16 *Как устроены уведомления в Telegram и какие у них риски?* Бот создаётся у *@BotFather*, *workflow* вызывает метод *sendMessage* с токеном бота и идентификатором чата. Токен лежит в секретах; он входит в адрес запроса, поэтому его нельзя печатать в журнал, а утёкший токен перевыпускается. Текст формируется из переменных окружения во избежание внедрения команд.

17 *Как не раскрыть секреты в CI и образах?* Не печатать значения, передавать их через *env*, а не аргументы команд, ограничить права токена (*permissions: contents: read*), не копировать *.env* в образ, пользоваться окружениями с защитой, регулярно перевыпускать ключи. Для запросов из форка секреты *GitHub* не передаёт, поэтому шаги деплоя в нём пропускаются.

ЗАКЛЮЧЕНИЕ

В ходе работы серверная часть сервиса планирования задач подготовлена к автоматическому развёртыванию и наблюдению. Добавлены линтер и 40 тестов, *Dockerfile* (образ собран и проверен локально), *workflow GitHub Actions* с проверками, сборкой образа, деплоем на *Render*, условным деплоем фронтенда на *Vercel* и уведомлениями в *Telegram*, конфигурации *render.yaml* и *vercel.json*.

Для наблюдения за работой реализованы проверки */health* и */health/ready*, защищённые */metrics* и */dashboard*, журналы с внешним транспортом *Better Stack*, отправка ошибок в *Sentry* с очисткой данных и безопасный маршрут тестовой ошибки. Всё, что можно проверить без внешних аккаунтов, проверено запуском; интеграции с *Sentry* и *Better Stack* проверены на локальном приёмнике.

Не выполнено (подробности по каждому шагу даны в разделе 2): создание аккаунтов и сервисов (*Render*, *Neon*, *Vercel*, *UptimeRobot*, *Sentry*, *Better Stack*, бот *Telegram*), добавление секретов в *GitHub*, первый запуск *workflow* и реальный деплой. Поэтому ссылка на работающее приложение и снимки внешних сервисов оставлены заглушками и будут добавлены после настройки.